

PROJECT WORK PACKAGE / 组员任务书

代码与数值管线负责人

职责: 建立模块化、可测试、可复现的 representation-to-circuit 实验管线，并在方法冻结后运行 selector、QAOA 与 noise 实验。

当前依据	URSS_project (9), 28 pages, compiled 4 Aug 2026
当前完成度判断	约 55%-65% 的代数与基础设施代码现在可稳定完成；最终 selector、compiler numbers、QAOA/noise 需等待 Chapter 4.5-4.7、5、6。
现在应做	现在完成 canonical PUBO、encoding、baselines、exactness tests、design enumeration、logical metrics、配置与数据框架；为 compiler/relaxation/QAOA 只搭接口。
暂时不要做最终版	不要自行补 candidate-pool 或 warm-start/mixer 定义；不要把 provisional compiler/QAOA 数字写成最终结果。

状态说明

现在可完成	当前 draft 已给出足够定义，可以直接完成并作为正式交付。
现在搭框架	现在建立稳定接口、数据结构和 pilot；相关章节冻结后重跑正式数据。
等待定义冻结	不能凭空补定义或提前产结果；先等待负责人冻结方法。

共同原则

- 所有结果必须可由固定 seed 和配置文件重现。
- 先验证定义与数据，再生成图；不要以几张看起来好的图反向选择参数。
- 遇到 draft 未定义的选择时，记录为 decision needed，不自行默认为最终方案。

1. 先读懂哪些子章

优先级	当前 draft 子章	读懂后的检查点
必读	2.1 Canonical pseudo-Boolean representation	实现唯一 canonical form、evaluation、degree、msupp 与 coefficient aggregation。
必读	2.3 Quadratzation and pointwise exactness	理解验证器必须检查 $f(x)=\min_y F(x,y)$ ，并区分 exact value 与 unique consistent fibre。
必读	2.4 Circuit resource measures	知道 logical width、gate/depth、SWAP、dynamic range 和 interaction graph 的定义边界。
必读	3.1 General one-step product substitution	实现 residual A_p 、Rosenberg block、threshold $\ A_p\ _\infty$ 与 strict/non-strict 测试。
必读	3.3-3.6 Cubic theory and exact design	实现 D/R 、 ρ 、 P_ρ 、 C_p^+/C_p^- 、sharp threshold、reuse、pair-cover 和 feasible design。
必读	3.7 and 7.1.1 worked examples	把它们变成 golden unit tests，逐项匹配 Eq. (78)/(125)/(126)。
必读	4.1-4.5 Selection	实现 endpoint encodings、resource vector、 C_{rel} formula 与 local algebraic updates；明确 $4m$ 应为 4^m 。
浏览	2.2, 2.5, Sections 8-9 headings	理解 Hamiltonian、relaxation 输出与最终实验指标，但暂不冻结 circuit。
等待	3.2; 4.6-4.7; Chapters 5-6	3.2 后补 arbitrary-degree tests；4.6-4.7 决定 search；5-6 决定 warm-start 与 compilation。

阅读检查点
先让程序复现 Section 7.1.1 的 canonical coefficients、三种 encodings、 $M_{23}=6$ 、 $M_{14}=3$ 和 32 个 x 上的 pointwise identity。通过后再扩展到随机实例。

2. 建议的代码结构

```
src/
pbo.py canonical polynomial and evaluator
penalties.py Rosenberg blocks and thresholds
designs.py sigma, D/R, rho, pair blocks
encodings.py native/selective/full transformations
baselines.py deterministic, min-ancilla, random
exactness.py exhaustive correctness checks
resources.py logical metrics
compiler.py interface now; final after Ch.6
selector.py interface now; final after Sec.4.6-4.7
relaxation.py interface and C_rel checks
qaoa.py interface now; final after Ch.5
experiments/ executable experiment entry points
configs/ versioned YAML/JSON configurations
tests/ unit, property, regression tests
data/{raw,processed}/ immutable inputs and derived tables
results/ machine-readable tables before figures
```

架构原则

每个 stage 输入和输出都可序列化; instance seed、design ID、compiler seed、optimizer seed 与 noise seed 分开保存。图表脚本只能读取 results tables, 不能重新计算隐藏逻辑。

3. 按顺序执行的任务

C0 建立 repository、配置和结果 schema [协作: Benchmark 负责人]

现在可完成

目标: 让所有后续实验有稳定且可追踪的输入输出。

- 为 instance、design、circuit 和 run 建立 deterministic IDs。
- 区分 raw data 与 derived data; raw instances 不允许被结果脚本修改。
- 所有可变参数进入 versioned config, 不使用 notebook 隐藏状态。
- 建立 test、lint、environment lock 和一键最小 smoke test。

完成标准: 从空环境运行 smoke test 能生成同一结果记录; 配置中包含全部 seeds 和软件版本。

C1 实现 canonical PUBO core [协作: Benchmark 负责人]

现在可完成

目标: 把任何输入 polynomial 转为唯一、可比较的 canonical representation。

- support 使用排序后的不可变结构; 重复 monomial 自动聚合, 零系数删除。
- 实现 evaluate、degree、msupp、C3(f)、pair shadow 与 coefficient lookup。
- 提供 Boolean assignment 的批量 evaluation 与 direct-vs-canonical cross-check。
- 处理 constant、linear、quadratic、cubic terms, 不假定输入已经排序。

完成标准: property tests 覆盖变量排列、重复项、零系数、随机 assignments; canonicalization 幂等。

C2 实现 exact encoding 与 penalty 模块

现在可完成

目标: 从 $\text{design } \sigma = (D, \rho, M)$ 构造 F_σ , 并计算 sharp thresholds。

- 实现 Rosenberg $P_R(a, b, y)$ 、 A_p 、 C_p^+ 、 C_p^- 、 T_p 和 $M_p = T_p + \epsilon$ 。
- 校验每个 reduced cubic support 的 $\rho(A)$ 必须是其 constituent pair。
- 只为实际使用的 pair 创建 auxiliary; 修订后的 z_p 约束需同步进入验证器。
- 输出 stable auxiliary ordering, 避免同一 design 因 dictionary order 改变 circuit。

完成标准: 能够精确复现 Sections 3.7/7.1.1, 并对非法 design 给出明确错误。

C3 实现代数层强基线 [协作: Benchmark 负责人]

现在可完成

目标: 提供公平且可重复的 endpoint/reference encodings。

- all-native、deterministic fully quadratized、no-sharing full。
- minimum-ancilla full: 通过 pair-cover/set-cover 求解或可靠算法, 保存 optimality 状态。
- matched-random selective: 匹配 auxiliary count 或 reduced-term count, 多 random seeds。
- hardware-aware full-QUBO 先保留接口, 等 compiler protocol 后实现。

注意: fully quadratized baseline 不能只是随意选择一组弱 pair assignments。

完成标准: 同一 seed 下 baseline design 相同; small cases 能与 exhaustive pair-cover optimum 对照。

C4 建立 exactness、sharpness 和 unique-fibre tests

现在可完成

目标: 将 Chapter 3 的数学声明变成自动回归测试。

- 对小实例枚举所有 x 和所有 auxiliaries y , 计算 max pointwise mismatch。
- 测试 $M = T - \delta$ 应出现 witness; $M = T$ 保留 objective 但允许 tie; $M = T + \epsilon$ 应 unique。
- 分别覆盖同号、mixed-sign、shared-pair 与 isolated-pair blocks。
- 随机生成数百组 cubic unit tests, 并保存失败 witness 而不是只返回 False。

完成标准: 所有 golden/random tests 通过; 报告 mismatch count、tie count 和 representative witnesses。

C5 实现 small-design enumeration 与 logical metrics

现在可完成

目标: 为未来 oracle comparison 建立完整的小规模设计数据库。

- 枚举 retained 或三种 constituent-pair choices; 注意 unrestricted termwise count 是 4^m 。
- canonicalise 并去除等价 designs; 记录 D 、 ρ 、 P_ρ 、 M 。
- 计算 auxiliaries、retained cubic count、quadratic couplings、 $M_{\max}$ 、 κ 与 graph statistics。
- 先生成 Pareto 数据; J_{res} 和 pool-relative winner 在 weights/pool 冻结后再算。

完成标准: 每个 small instance 有可查询 design table, 能返回 endpoints、Pareto set 和 logical metrics。

C6 建立 compiler harness 与正确性测试

现在搭框架

目标: 先隔离接口和缓存, 避免 Chapter 6 完成后重写整个 pipeline。

- 接口输入为完整 design/cost layer; 输出 compiled circuit 与统一 resource record。
- 预留 architecture、basis gates、layout/routing、optimization level 和 seed。
- 实现小系统 unitary/diagonal-phase comparison, 分别检查 native cubic 和 penalty blocks。
- 缓存键必须包含所有 compiler configuration; 不要只缓存 design ID。

注意: selector 不能挑中一个幸运 compiler seed; 最终需 train/held-out seeds 或多 seed aggregation。

完成标准: 现在完成 API、mock/pilot 与 unit tests; 正式 G_{2q}、D_{2q}、SWAP 数字在 Chapter 6 冻结后全部重跑。

C7 实现 candidate-pool selector

等待定义冻结

目标: 在方法冻结后实现可复现的 greedy/beam search。

- 等待 block proposals、forward/backward/swap/reassignment、beam width 与 multistart 定义。
- 等待 stopping、tie-break、hard budget、cache 与 eta 规则。
- 区分 global sigma*、full-space oracle、candidate-pool optimum 与 heuristic output。
- 随后在 small instances 报告 hit rate、regret、Pareto recall、compiler calls 与 runtime。

完成标准: 4.5-4.7 形成完整伪代码并经负责人确认后才开始最终实现; 现在只定义抽象 SearchStrategy 接口。

C8 实现 relaxation 与 C_{rel} 验证 [协作: Benchmark 负责人]

现在搭框架

目标: 先验证公式和数据接口, 再等待 Chapter 5 决定实际 relaxation/warm start。

- 接口同时允许 first moments μ_i 与 pair moments μ_{ij}/q_p 。
- 实现 clipping、 ν_p 、 ℓ_p 、C_{rel}, 并用 Monte Carlo 检查期望公式。
- 保存 solver、tolerance、status、objective 与 moments, 不只保存 μ 。
- 预留完整 expected encoding distortion 作为诊断, 避免把 C_{rel} 误写为完整误差。

完成标准: 公式测试现在可完成; solver choice、 q_p construction 和正式 score 等 Chapter 5/方法冻结。

C9 实现 warm-start QAOA 与 noise pipeline

等待定义冻结

目标: 在 Chapters 5-6 冻结后生成可公平比较的量子结果。

- 等待 initial state、mixer、parameter initialization、 p 、optimizer、restarts、shots 与 decoding。
- 所有 encodings 独立优化参数, 但 optimizer evaluations/restarts/shots 相同。
- 测量后丢弃 auxiliary bits 并用原始 $f(x)$ 评分; 另报 encoded energy 和 inconsistency。
- 同时实现 equal- p 与 equal-compiled-2Q-budget comparisons; 主结果不 postselect。

注意: pointwise exactness 不保证不同扩展 Hilbert spaces 上的 QAOA dynamics 相同。

完成标准: 协议写入 config 并冻结后再产最终结果; 现在只搭类接口与一个非论文用途 smoke test。

C10 运行 ablations、scaling 与 regime map [协作: Benchmark 负责人 + 项目负责人]

等待定义冻结

- 目标: 把完整 pipeline 的结果转化为可推广结论。
- Ablations: resource-only、C_rel off、compiler proxy、sharing off、sharp vs conservative penalty、greedy vs beam、cold vs warm。
 - Sensitivity: eta、epsilon、weights、clipping delta；特别检查 C_rel(native)=0 的偏置。
 - Scaling: QAOA 用小 n；compiler/search 用更大 n；报告 wall time、memory、compiler calls。
 - Regime map 在 held-out instances/topologies 上验证 selective 的 win/tie/loss。
- 完成标准: 所有主图可由一个命令从 frozen result tables 重画，并附 paired uncertainty intervals。

4. 协作、注意事项与交付

- [协作节点]
1. 数据契约: Benchmark 负责人定义字段语义；你实现 schema、validator 和 canonical evaluator。
 2. Ground truth: 对方给出 direct evaluator；你运行 brute-force/MILP 并回传 optimum、degeneracy、gap 和 status。
 3. Regime map: 对方解释问题特征；你负责 extraction、paired statistics、held-out evaluation 和 figures。

必须避免的实现错误

- 不要比较不同 encodings 的 encoded energy F_sigma 作为共同主指标；共同指标是原始 f(x)。
- 不要在 test instances 上调 weights、eta、beam width、compiler seed 或 optimizer settings。
- 不要让 auxiliary ordering、dictionary ordering 或 compiler cache 造成非确定性。
- 不要只报告 best run；保存每个 instance/seed/restart 的原始记录。
- 不要把 interface stub 产生的 pilot 图写入最终 Results。

交付物	形式	何时可定稿
Core library	canonical PUBO, designs, encodings, penalties	现在可完成
Correctness suite	golden + random exactness/tightness tests	现在可完成
Baselines and oracle data	native/full/min-ancilla/random + small design tables	代数层现在；hardware-aware 后期
Compiler module	API, cache, correctness tests, resource records	现在搭框架；Chapter 6 后重跑
Selector module	greedy/beam/pool/oracle validation	等待 4.5-4.7
Relaxation/QAOA module	moments, warm start, optimizer, noise, decoding	接口现在；Chapters 5-6 后完成
Reproduction package	configs, seeds, raw/processed tables, one-command figures	框架现在；随实验持续填充

- 第一轮建议提交
- (1) repository skeleton; (2) canonical PUBO + encoding modules; (3) worked-example regression test; (4) random exactness suite; (5) small-design logical table; (6) compiler/relaxation/QAOA interfaces，不含最终数值。